# AURORA: Artificial Unified Reasoning and Operational Response Assistant

Project report | AURORA v1.0.0 | November 2026


## Abstract

AURORA is a modular, locally run AI assistant. It answers questions with a local large language model served by Ollama, grounds answers in a curated knowledge base through retrieval-augmented generation (RAG), searches the web with source citations, uses small tools such as a calculator and clock, and keeps conversational memory within a session. It exposes a command-line chat, a FastAPI backend with a streaming endpoint, and a browser interface. This report documents AURORA's motivation, architecture and request lifecycle; describes the engineering review and hardening carried out on the codebase (bug fixes, security limits, a Windows-compatible embedding backend, follow-up question handling and an expanded knowledge base); and states its limitations plainly. The system is a single-user tool intended to run on a developer's own machine.


## 1. Introduction

Cloud assistants are convenient but send prompts and documents to third parties, depend on connectivity and offer little control over behaviour. Local models remove those constraints, yet a bare model has three weaknesses: it knows nothing about the user's own documents, its knowledge stops at its training date, and it cannot calculate or look things up reliably. AURORA addresses these by composing a local model with retrieval, web search and tools behind a single routing layer, so each question is handled by the most suitable capability.

## 2. Objectives and scope

**Objectives.**

1. Run entirely on the user's machine with open models (Ollama).
2. Answer from a knowledge base when the question is covered, with grounded, cited web answers for current information.
3. Route each request to the right capability automatically.
4. Keep the codebase modular, testable and easy to extend.

5. Provide a usable interface: command line, REST API and web UI.

**Out of scope.** Multi-user hosting, authentication, horizontal scaling, fine-tuning models, and long-term cloud storage.

## 3. System overview

A single coordinating class, `AuroraEngine`, owns the model, tools, memory, session and retrieval components. A request passes through these stages:

**User message → follow-up resolver → capability router → orchestrator → executor → (knowledge base | web | tool | model) → result synthesizer → streamed answer**

| Package | Responsibility |
| --- | --- |
| `app/engine` | `AuroraEngine`: wires components and exposes chat, streaming chat and event streaming |
| `app/routing` | Capability router: decides whether a request needs a tool, the knowledge base, the web or the plain model |
| `app/orchestration`, `app/execution` | Plans the capabilities for a request, executes them, and synthesizes results; supports multi-intent requests |
| `app/rag`, `app/vectorstore` | Document loading, chunking, embeddings, FAISS vector search, query rewriting, relevance filtering, grounded generation and evaluation |
| `app/web` | Web search, temporal query rewriting, relevance scoring, result processing, source grounding with citation identifiers, and answer validation |
| `app/tools` | Tool base class and registry; calculator, date, time and web-search tools |
| `app/memory`, `app/session` | Memory extraction, conversation summaries and JSON session storage |
| `app/models`, `app/prompts` | Ollama and test models; system prompt and prompt construction |
| `backend` | FastAPI application and the static web interface |

# 4. Component design

## 4.1 Capability routing and orchestration

The router applies ordered rules (calculation, date and time, web, tools) and falls back to the RAG router, which sends a request to the knowledge base only when it names a known topic **and** expresses a knowledge intent (for example "what is", "explain", "who created") without a generation intent (for example "write", "build"). The orchestrator sits above the router: it detects requests that need several capabilities, prioritises the web for time-sensitive questions, and preserves streaming for single-capability requests.

## 4.2 Retrieval-augmented generation

The knowledge base is a plain text file. Documents are split into overlapping word-based chunks (60 words with 15 words of overlap), embedded, and stored in a FAISS index. For each question, the query rewriter expands abbreviations (for example ML to machine learning), the retriever returns the nearest chunks, a relevance checker discards chunks beyond a distance threshold, and the generator answers using only the retained context. If nothing relevant is found, the assistant is not forced to guess from irrelevant text.

## 4.3 Embeddings

Embeddings are produced by Ollama by default (model `nomic-embed-text`), with an optional Hugging Face backend selected by configuration. The vector size is discovered at runtime, so changing the embedding model requires no code change.

## 4.4 Web intelligence

Web answers use a pipeline of query rewriting (including resolving references such as "it" from conversation history), relevance scoring, result processing and source grounding. Each retained source receives a citation identifier such as `[1]`, and a validator checks release and version claims against the evidence before they reach the user. The interface shows sources as cards above the answer.

## 4.5 Tools

The calculator evaluates arithmetic through Python's `ast` module rather than `eval`, permitting only numeric literals and arithmetic operators. Date and time tools return the local clock. Tools register through a common base class so new ones can be added without changing the engine.

## 4.6 Memory and sessions

A memory extractor identifies facts worth keeping, a summary manager condenses older messages once the history exceeds a configured window, and the session manager persists history to JSON on disk.

## 4.7 Interfaces

The web interface streams answers token by token over newline-delimited JSON events and supports stopping a response, copying answers, clickable prompts and a live online indicator. The API is documented in Appendix A.

## 5. Worked example: follow-up questions

A follow-up such as "what else do you know about it?" contains no topic, so it cannot be routed or searched on its own. AURORA's follow-up resolver remembers the last knowledge topic and rewrites short follow-ups before routing:

| User message | Resolved message | Route |
|---|---|---|
| what is python | what is python | knowledge base |
| what else do you know about it | what else do you know about python | knowledge base |
| tell me more | tell me more about python | knowledge base |
| what is faiss | what is faiss | knowledge base |
| how does it work | how does faiss work | knowledge base |
| what time is it | what time is it (unchanged) | time tool |

The resolver acts only on short messages that use a pronoun or a phrase such as "tell me more", ignores time-related questions, and forgets the topic after four unrelated messages.

## 6. Engineering review and validation

The codebase (about 9,000 lines of Python with 38 test files) was reviewed by reading the source and running the automated suite. The table summarises the defects and weaknesses found and how each was addressed.

| Finding | Cause | Resolution | Verification |
|---|---|---|---|
| Calculator could hang on inputs such as `9**9**9**9` | Exponentiation was unbounded | Limits on exponent size, result size and expression length | 6 regression tests |
| Crash when web answers had no intelligence data | A variable was assigned inside a conditional block but used outside it | Default value before the block | Previously failing tests now pass |

| Finding | Cause | Resolution | Verification |
|---|---|---|---|
| Evidence labelled `[[1]]` in web prompts | Citation identifiers were already bracketed and were bracketed again | Brackets added only when missing; test updated to guard against recurrence | Regression test |
| Orchestrator tests failing | A test double had not been updated when the executor gained a streaming option | Test double updated | 6 tests pass |
| API accepted any input and could leak errors | No validation or error handling | Message length limits (1 to 4000 characters), generic error responses, error events in the stream | Reviewed; exercised manually |
| Application failed to start on Windows | An Application Control policy blocked PyTorch's DLL, needed only for embeddings | Ollama-based embeddings as the default; Hugging Face made optional | 5 unit tests; start-up confirmed by the author |
| Follow-up questions had no answer | Router required a topic word in the same message; the knowledge base held four sentences | Follow-up resolver, whole-word router matching, new intents, and an expanded knowledge base | 14 tests |
| Repository would have published private data | Empty ignore file; log, chat history and environment file untracked | Ignore rules, environment template, cleaned history | File-list review before publishing |

The suite collected 41 tests and all passed on the author's machine after the first set of fixes. Later changes added regression tests for the calculator limits, the Ollama embeddings and the follow-up resolver. A continuous-integration workflow compiles the code and runs the fast tests on every push.

## 7. Security considerations

- **Local by design.** Prompts, documents and chat history stay on the user's machine.
- **Safe expression evaluation.** The calculator parses expressions with `ast` and now bounds their cost.
- **Indirect prompt injection.** Web pages and documents are passed to the model as context. Hidden instructions in that content can attempt to influence answers. Source grounding and validation reduce but do not eliminate this risk.
- **No authentication.** The API has no login or rate limiting and must stay bound to `127.0.0.1`.
- **Data hygiene.** Environment files, logs and chat history are excluded from version control.

## 8. Setup summary

Requirements: Python 3.10 or newer, Git and Ollama. Install the dependencies in a virtual environment, copy `.env.example` to `.env`, pull a chat model and the embedding model with Ollama, then run `python -m backend.api` and open `http://127.0.0.1:8000`. The README lists the exact commands and troubleshooting for common errors.

## 9. Limitations

1. A single global engine and chat history serve every API client, so AURORA is a single-user tool.
2. Routing relies on topic and intent word lists, not semantic understanding, so unlisted topics are not sent to the knowledge base.
3. The knowledge base is small and hand-written; answer quality is bounded by its contents.
4. The relevance distance threshold was tuned for the original embedding model and may need adjustment (configurable) after changing embeddings.
5. The follow-up resolver keeps one global topic for up to four turns.
6. The engine file is large (about 1,300 lines) and would benefit from being split.
7. Some tests require Ollama or downloaded models and are not yet isolated.
8. A release-decision instruction is added to every web answer, including questions unrelated to software versions.
9. No formal evaluation of answer accuracy has been performed.

## 10. Future work

Per-user sessions and authentication; semantic (embedding-based) routing instead of word lists; document upload and automatic re-indexing; automatic skipping of Ollama-dependent tests; a retrieval-quality benchmark with measured accuracy; splitting the engine into smaller services; and adversarial testing of the web and document pipeline for prompt injection with a red-team tool such as ZEROTH.

## 11. Conclusion

AURORA demonstrates that a useful assistant can be built entirely from local components by composing retrieval, web grounding and tools behind a routing layer. The review and hardening described here removed several real defects, made the system start on locked-down Windows machines, and made multi-turn conversation work with the knowledge base. Its limitations are those of a single-user prototype, and each has a clear next step.

## Appendix A. API reference

| Method | Path | Purpose |
|---|---|---|
| GET | `/` | Web interface |
| GET | `/health` | Liveness check |
| GET | `/status` | Name, version and environment |
| POST | `/chat` | Body `{"message": "..."}` (1 to 4000 characters); returns the full answer |
| POST | `/chat/stream` | Same body; returns newline-delimited JSON events |

Stream event types: `status` (progress text), `sources` (list of cited sources), `answer` (a chunk of the answer), `done`, and `error`.

# Appendix B. Configuration

| Variable | Meaning |
|---|---|
| `APP_NAME`, `APP_VERSION`, `ENVIRONMENT`, `DEBUG` | Application identity and mode |
| `MODEL_NAME` | Chat model served by Ollama |
| `EMBEDDING_PROVIDER` | `ollama` (default) or `huggingface` |
| `EMBEDDING_MODEL` | Embedding model name (default `nomic-embed-text`) |
| `RAG_MAX_DISTANCE` | Relevance distance threshold (default 1.0) |
| `HOST`, `PORT`, `LOG_LEVEL`, `CONTEXT_WINDOW` | Server and memory settings |

# Appendix C. Glossary

| Term | Meaning |
|---|---|
| RAG | Retrieval-augmented generation: answering from retrieved documents |
| Embedding | A vector of numbers representing the meaning of text |
| FAISS | A library for fast nearest-neighbour search over vectors |
| Chunk | A short overlapping piece of a document that is embedded and retrieved |
| Grounding | Tying an answer to cited sources |
| Prompt injection | Text that tries to override an assistant's instructions |
| NDJSON | Newline-delimited JSON, one event per line |

# References

1. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
2. Johnson, J., Douze, M. and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.
3. OWASP Foundation. *OWASP Top 10 for LLM Applications* (2025 edition), LLM01 Prompt Injection.
4. Ollama documentation. Running large language models locally.
5. FastAPI documentation.